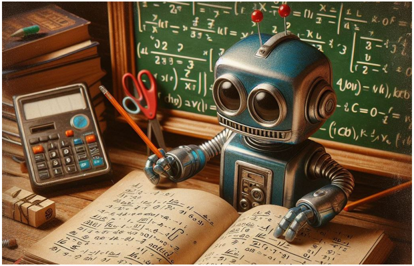
Overview of Scientific Machine Learning

LAGA seminary - MSC team

Antoine Bensalah – Joël Soffo
Airbus Central R&T

February 22, 2026



The XRV team and its main modeling subjects

Central Research & Technology

Gilles Peres

Electrification

Enable the future of hybrid Electric flight by pushing the limit of electrical technologies while mastering integration.

- Electrical motors
- Power Electronics
- Electrical Power Distribution
- Storage Elements
- Fuel Cells

Volker Ziegler

Communication Technologies

One-stop shop for solving communication and embedded computing challenges.

- RF, Optical and Wired Technologies
- Comms Systems & Networks
- Advanced Sensors Technologies
- Secure and Safety-critical Comms
- Onboard Computing Architectures

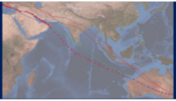
Christian Weimer

Materials

Pushing for sustainable aviation, enhanced mission/product life performance, competitiveness for in-/organic materials classes incl. their processes and surfaces.

- Sustainability & Circularity
- Functionality
- Digital Materials
- Performance Materials

Jayant Sen Gupta

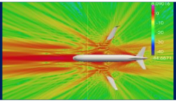


Artificial Intelligence

Pushing and maturing AI technologies to support their safe and efficient integration into our products, systems and operations.

- Trustworthy AI
- Scale AI
- Frugal AI
- Adaptive Human Interaction

Yannick Deleris



Virtual Product Engineering

Anticipate and optimise business performance through upstream research in modelling and simulation for our products, systems and operations.

- Systems Engineering
- Modeling
- Scientific Computing for M&S
- Appl. Mathematics

About us

Antoine Bensalah

- 2018: PhD at POEMS (ENSTA, Inria, CNRS)
  - Patrick Joly, Jean-François Mercier
  - mathematical aspects of a propagation aeroacoustic model (Goldstein's model)
- Since 2018: Airbus Research Engineer
  - Acoustic modelling and simulation
  - For 3 years: SciML: operator learning, PINN, ...

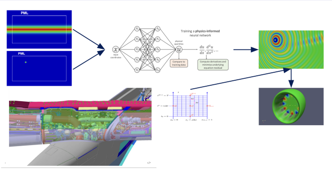
Joël Soffo

- Internship at Airbus in 2022
- Cifre PhD contract since 2023
- Academic supervisor: Anthony Nouy (Jean Leray Lab, Univ Nantes - Centrale Nantes)

Two Airbus CR&T projects: MOTO / Concace

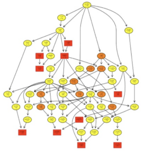
MOTO – ”Modelling of Tomorrow”

- **Goal**: Hybridization Scientific Computing & Machine Learning
  - methodology monitoring, build database for benchmark
  - watch / stimulate start-up and commercial software
- **External partners**: Centrale Nantes, Sorbonne University, Inria, Neural Concept, Phimeca



Concace: Numerical & Parallel Composability for HPC

- **”Classical” HPC**: H-matrix task-based solver, ...
- Also **machine learning based HPC**
  - Learning of preconditioners for Helmholtz
  - Learning of matrix rank (for $\mathcal{H}$-matrix parallelization)
- **External partners**: Inria, Cerfacs, Onera, LIP6, IRT SystemX



AIRBUS
CERFACS
Inria

The place of physics modelling in Airbus

Design exploration	Validation	Certification
weak error control	strong error control	High fidelity solution
$\rightarrow$ inductive charging	$\rightarrow$ acoustic liners	$\rightarrow$ antenna pattern

$\rightsquigarrow$ **Goal of our team**: understand the mathematical tools of modelling, propose methodologies, spread them through Airbus

Major stakes of AI

- A lot of money invested and to be invested
- A new field for research

Airbus needs

- Do not miss the opportunity ...
- Keep upgrading its capabilities to not left behind
- Do not let be sold it anything

Outline

2 - Theoretical approach of meta-model in high dimension

Context of approximation theory

$(V, \|\cdot\|)$ a Banach / Hilbert space of functions $\Omega \subset \mathbb{R}^d \to \mathbb{R}$.

Goal: Find an approximation space $V_n \subset V$ (n related to a complexity) to represent functions $u : \Omega \to \mathbb{R}$ from V,

$$\text{find } \; u_n \in V_n \quad / \quad u_n \approx u.$$

What kind of information is known about u?

- Point evaluation: sample $\{u(x_1), \ldots, u(x_N)\}$
- Equation satisfied by u (PDE context)
- More generally: linear "measurements" of u

$$\ell_i : u \in V \mapsto \ell_i(u) \in \mathbb{R}, \quad (1 \leq i \leq N)$$

Typical questions of approximation theory

Theoretical error decay: error of best approximation

$$e_n(u) := \inf_{u_n \in V_n} \|u - u_n\|_V$$

- Universality ? $e_n(u) \to 0$, as $n \to +\infty$
- Approximation classes: $\gamma : \mathbb{N}^* \to \mathbb{N}^*$ a growth function (increasing)

$$\mathcal{A}^\gamma := \left\{ u \in V \,/\, \sup_n \gamma(n)\, e_n(u) < \infty \right\}$$

Build an algorithm

- How to practically build a $\hat{u}_n \in V_n$, which is (quasi) optimal?
- At which complexity (in n)?

Encoder / Decoder point of view – notions of n-widths

Useful point of view

Encoder: $\begin{array}{cccc} E_n : & V & \to & \mathbb{R}^n \\ & u & \mapsto & (c_k) \end{array}$ **Decoder:** $\begin{array}{cccc} D_n : & \mathbb{R}^n & \to & V_n \subset V \\ & (c_k) & \mapsto & u_n \end{array}$.

- Approximation space: $V_n = \mathrm{Im}(D_n)$, $u_n := D_n \circ E_n(u) \approx u$
- E_n: the "algorithm" which builds *latent coefficients* $(c_k)_{1 \le k \le n}$ from information about u!
- D_n: construction of the approximation ("reconstruction of u") from $(c_k)_{1 \le k \le n}$.

General n-widths: $\mathcal{K} \subset V$ (compact)

$$\mathrm{width}_n^{\mathrm{wc}}(\mathcal{K}) := \inf_{E_n, D_n} \sup_{u \in \mathcal{K}} \| u - \underbrace{D_n \circ E_n(u)}_{=u_n} \|_V$$

$E_n \setminus D_n$	linear	non linear
linear	approx. num.	sensing number $s_n(\mathcal{K})$
non linear	Kolmogorov $d_n(\mathcal{K})$	manifold widths $\delta_n^L(\mathcal{K})$ $(*)$

$(*)$ one restricts ourselves to L-Lip E_n, D_n for stability and avoid pathological cases!

Curse of dimensionality

Asymptotic in complexity n

In $V := L^p([0,1]^d)$, $1 \leq p \leq \infty$, $\mathcal{B}$ unit ball of $W^{k,p}([0,1]^d)$

Linear tools: $d_n(\mathcal{B}) \sim_{n\infty} n^{-k/d}$ **Nonlinear tools:** $\delta_n^L(\mathcal{B}) \sim_{n\infty} n^{-k/d}$

With extra regularity ?

Tractability notion: d-dependence the estimations: $c(d)\,n^{-k/d} \leq d_n(\mathcal{B}) \leq C(d)\,n^{-k/d}$?

Tractability issues

In $V := L^\infty([0,1]^d)$, infinite smoothness: $\mathcal{K} := \left\{u \in \mathscr{C}^\infty([0,1]^d) \,:\, \sup_\alpha \|D^\alpha u\|_{L^\infty} < \infty\right\}$
Even if $d_n(\mathcal{K}) \sim n^{-k}$ for all $k \geq 1$ ...

Linear tools
$\min\left\{n \geq 0 \,:\, d_n(\mathcal{K}) \leq 1/2\right\} \geq C \times 2^{d/2}$

Nonlinear tools
$\min\left\{n \geq 0 \,:\, s_n(\mathcal{K}) \leq 1\right\} \geq 2^{\lfloor d/2 \rfloor}$

What is about the Neural Network? Positive results

Universal property of shallow neural networks: "$e_n(u) \to 0$"

Density in $\mathscr{C}([0,1]^d)$ if σ is not a polynomial:

$$\overline{\text{span}}\left\{x \mapsto \sigma(w^T x + b) \,:\, w \in \mathbb{R}^d, b \in \mathbb{R}\right\} = \mathscr{C}([0,1]^d;\mathbb{R})$$

Historical papers: Cybenko (1989), Hornik (1991), Pinkus et al. (1993)

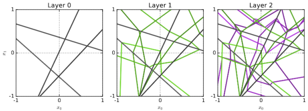
Beyond the classical smoothness classes

Importance of depth (compositions): depth creates

- an exponential number of "oscillations" of the function
- piecewise linear function on an exponential number of "cells" partitioning the space

$\rightsquigarrow$ Deep Neural Networks approximate with few parameters: Takagi functions, fractals, ...

From *"On the expressiveness of Deep Neural Networks"*, Raghu, Poole, Kleinberg, Ganguli, Sohl Dickstein (2017).



What is about the Neural Network? But...

The curse of dimensionality cannot be outpassed!

But DNN optimization is very difficult!! *Theory to practice gap*!

- *"The gap between theory and practice in function approximation with deep neural networks"*, Adcock, Dexter (2021)
- *"Proof of the theory to practice gap in deep learning via sampling complexity bounds for Neural Network approximation spaces"*, Grohs, Voigtlaender (2023)

Practical studies show

$\rightsquigarrow$ For smooth functions: NNs perform poorly ...

$\rightsquigarrow$ For rough functions: theoretical rate of convergence is not attained!

Conclusion: a lot of challenges for scientific computing!

- Stability of DNNs
- Optimization of DNNs
- Sample complexity: for PDE applications, million / billion examples are not available!!
- Computation resources: state of the art deep learning model for image of language cost million dollars to train! (CO_2 emissions, ...)

3 - Our problem class of interest: parametric PDEs

3 - Our problem class of interest: parametric PDEs

The abstract parametric PDEs problem setting

General settings

- A parameter set $\mathscr{P} \subset \mathbb{R}^p$
- $\Omega \subset \mathbb{R}^d$, where $d \in \{1,2,3\}$ and a Hilbert space $(V, \|\cdot\|)$ of functions: $V \subset \mathscr{F}(\Omega;\mathbb{R})$

Abstract parametric problem (P_μ)

Given $\mu \in \mathscr{P}$, find $u(\mu) \in V$ such that

$$(\mathrm{P}_\mu): \quad \mathrm{L}(u(\mu);\mu) = f(\mu) \ \text{ in } \ V',$$

- $\mathrm{L}(\cdot;\mu) : V \to V'$ is a linear functional (represents the PDEs operator)
- $f(\mu) : V \to \mathbb{R}$ is a linear form (the source term)

Applications: multi-query problems

- **Optimization:** the best μ for a given purpose
- **Uncertainty quantification:** μ parametrizes a random field (Karhunen Loeve decomposition), compute $\mathbf{E}\big[Q(u(\boldsymbol{X}))\big]$, $\mathbf{P}\big[Q(u(\boldsymbol{X})) > \alpha\big]$, ...
- **Inverse problem:** given some measurements of u, to which μ it corresponds?

Examples

Prototypical second order elliptic equation

$$(P_\mu) \; : \; \begin{cases} -\mathrm{div}\big[a(\mu;\cdot)\nabla u(\mu)\big] = f, & x \in \Omega, \\ u(x) = 0, & x \in \partial\Omega. \end{cases}$$

with the parametrization

$$a(\mu;x) = \sum_{j=1}^{Q} \alpha_j(\mu)\, a_j(x) \quad \mu \in \mathscr{P}, \; x \in \Omega$$

"Goldstein's" toy case

Parameter: $\mu := (k, m)$,

$$(P_{k,m}) \; : \; \begin{cases} -ik\psi(m;x) + m\dfrac{d\psi(m;\cdot)}{dx}(x) = f(x), & x \in (0,1), \\ \psi(0) = 0. \end{cases}$$

Practical setting

Prerequisites

Have a reference code (high-fidelity solver) which provides the discrete solution map

$$\mu \in \mathscr{P} \mapsto u_h(\mu) \in V_h$$

$$V_h \subset V, \qquad 1 \ll \dim V_h =: N_h < \infty, \qquad \text{(typically } V_h \text{ a finite element space)}$$

Approximated *solutions manifold*

$$\mathscr{M}_h := \left\{u_h(\mu) \,:\, \mu \in \mathscr{P}\right\} \subset V_h, \qquad \mathscr{M}_h \approx \mathscr{M} := \left\{u(\mu) \,:\, \mu \in \mathscr{P}\right\} \subset V.$$

Goal

Learn the *solution map*

$$\mu \in \mathscr{P} \mapsto u_h(\mu) \in V_h$$

What is known about the solution $u_h(\mu)$?

- *snapshots* $\left\{u_h(\mu^1), \dots, u_h(\mu^N)\right\} \subset V_h$
- We have a high fidelity solver $\Rightarrow$ μ^j can be chosen adaptively!

3 **Our problem class of interest: parametric PDEs**

- Parametric PDEs settings
- The Reduced Basis Methods approach
- Nonlinear extension of the Reduced Basis Methods

The Reduced Basis Method approach

Approach: use the simplest approximation tools

$\rightsquigarrow$ linear encoder and decoder

Main ideas

- **Encoder:** $E_N : V_h \to \mathbb{R}^N$:

$$\mathbf{u}_N := E_N(u_h) \in \mathbb{R}^N,$$

- **Decoder:** $D_N : \mathbb{R}^N \to V_N \subset V_h$. Given a reduced space V_N

$$V_N := \mathrm{span}\{u_h(\mu^1), \dots, u_h(\mu^N)\} = \mathrm{span}\{\xi^1, \dots, \xi^N\} \subset V_h$$

  D_N provides the linear combination

$$u_N := D_N(\mathbf{u}_N) = \sum_{j=1}^{N} \mathbf{u}_{N,j} \xi^j \in V_N \subset V$$

$\rightsquigarrow$ One expects

$$u_N \approx u_h$$

Main questions

Questions

1. For what problems a low dimensional good reduced space V_N, with $N \ll N_h$, exists?
2. How to build such a reduced space V_N?
3. **Encoder:** How to devise a reduced problem to find $u_N(\mu) \in V_N$ approximating $u_h(\mu)$, with error bound guarantee, quasi-optimality?
4. How to solve in practice the reduced problem $(P_{\mu,N})$ without seeing the high fidelity dimension N_h?

Answers

1. Study the Kolmogorov n-widths decay: $n \mapsto d_n(\mathscr{M}_h)$
2. SVD/POD algorithms or Greedy algorithms $\rightsquigarrow V_N$
3. Solve a (Petrov) Galerkin projection problem $(P_{\mu,N})$ on the reduced space V_N
4. Efficient for affine decomposed problem. If not $\rightsquigarrow$ Empirical Interpolation Method (EIM)

$\rightsquigarrow$ Let us detail a bit $1)$ and $3)$

Question $1)$: RBM for which class of parametric PDEs ?

We are going to see three cases

- **The uniformly coercive affine/separable case**
  see *"Reduced Basis Methods: Success, Limitations and Future Challenges"*, Ohlberger, Rave (2016)
- **Holomorphic dependence of the solution**
  "Kolmogorov widths under holomorphic mappings", Cohen, DeVore (2015)
- "Breaking the curse of dimenionsality" with non linear tools: **Sparse polynomial approximation**
  "Breaking the curse of dimensionality in sparse polynomial approximation of parametric PDEs", Chkifa, Cohen, Schwab (2015)

Question $1)$: theoretical guarantee? 1/3

Variational formulation of the (abstract) problem:

$$(P_\mu): \text{ Find } u(\mu) \in V \,/\, \forall v \in V,\ a(u(\mu), v; \mu) = f(v)$$

Suppose that $a(\cdot,\cdot,\mu)$ is **uniformly coercive** and a is**affine decomposable** / **separable**:

$$a(\cdot,\cdot;\mu) = \sum_{q=1}^{Q} \theta_q(\mu)\, a_q(\cdot,\cdot)$$

- $\theta_q : \mathscr{P} \to \mathbb{R}$ continuous (weak assumption!) on $\mathscr{P} \subset \mathbb{R}^p$ compact
- $f : V \to \mathbb{R}$ continuous linear form and $a_q : V \times V \to \mathbb{R}$ continuous and coercive bilinear form

Theorem

Then, there exists constants $C, c > 0$

$$d_N(\mathscr{M}) \leq C \exp\left(-cN^{1/Q}\right)$$

Curse of dimension

$\rightsquigarrow$ estimation degenerates for $Q \to +\infty$!

X, Y Banach spaces,

$$u : O \subset X \longrightarrow Y \quad \text{holomorphic},$$

where $\mathscr{O}$ open of X, and s.t.

$$\sup_{\mu \in \mathscr{O}} \left\| u(\mu) \right\|_Y < \infty.$$

Theorem

If $\mathscr{P} \subset \mathscr{O}$ is compact, for any $s > 1$ and $\varepsilon > 0$ s.t. $s - 1 - \varepsilon > 0$,

$$\sup_{n \geq 1} n^s \, d_n(\mathcal{P}) < \infty \quad \Rightarrow \quad \sup_{n \geq 1} n^{s-1-\varepsilon} \, d_n\big(\mathscr{M}\big) < \infty$$

where

$$\mathscr{M} := \left\{ u(\mu) \, : \, \mu \in \mathcal{K} \right\}$$

- holomorphy of $u \Rightarrow$ almost none complexity is added to $\mathscr{P}$
- Applications to: $a \in L^\infty(\Omega) \mapsto u(a) \in H_0^1(\Omega)$
  - $-\mathrm{div}(a \nabla u) = f$, on bounded Lip domain $\Omega \subset \mathbb{R}^d$, with homogeneous Dirichlet b.c., $f \in H^{-1}(\Omega)$
  - $-\mathrm{div}(\exp(a) \nabla u) + u^3 = f$, bounded Lip domain $\Omega \subset \mathbb{R}^d$, $d = 2, 3$, homogeneous Dirichlet b.c., $f \in H^{-1}(\Omega)$

Question $1)$: theoretical guarantee? 3/3

$$\mu \in \mathscr{P} := [-1,1]^{\mathbb{N}^*},\ u(\mu) \in H_0^1(\Omega)\ /\ -\mathrm{div}\left[a(\mu)\nabla u(\mu)\right] = f, \quad \text{in } \Omega$$

$f \in H^{-1}(\Omega)$ and $\overline{a}, (\psi_j)_{j\geq 1} \in L^\infty(\Omega)$

$$a(x;\mu) := \overline{a}(x) + \sum_{j=1}^{+\infty} \mu_j \psi_j(x) \quad \text{s.t. } 0 < a(x,\mu) < R < \infty \quad \text{(uniform ellipticity)}$$

Theorem

If $(\|\psi_j\|_{L^\infty})_j \in \ell^p$, for some $p \in (0,1)$,

$$\sup_{\mu \in \mathscr{P}} \left\| u(\mu) - \sum_{\nu \in \Lambda_n(a)} \frac{1}{\nu!} \partial^\nu u(0) \mu^\nu \right\|_{H^1} \leq C n^{-s}, \quad \text{where } s = 1/p - 1 > 0.$$

$\Lambda_n(a)$: n multi-indexes of the largest terms of the Taylor expansions of a.

- **Non linear encoder** and **non linear decoder**!
- **the curse of dimensionality is broken**: infinite dimension in input!

Question $3)$: compute the encoder

The encoder $E_N : V_h \longrightarrow \mathbb{R}^N$: coefficients $\mathbf{u}_N \in \mathbb{R}^N$ from a new problem (P_μ)

Method: Galerkin projection $\rightsquigarrow$ the Reduced Problem

$$\begin{cases} \text{Given } \mu \in \mathscr{P}, \\ (\mathrm{P}_{\mu,N}): \ \text{Find } u_N(\mu) \in V_N, \ \text{s.t. } \forall w_N \in W_N, \ a(u_N(\mu), w_N; \mu) = f(w_N; \mu) \end{cases}$$

$\rightsquigarrow$ Either $W_N = V_N$ or built from it: $W_N = \{T_N v \ : \ v \in V_N\}$

$\rightsquigarrow$ From then high-fidelity problem: coercivity can be inherited, **not the inf-sup stability!**

Quasi-optimal oblique projection

Linear system: find $\mathbf{u}_N = E_N(u_h(\mu))$ solution of

$$\mathbf{A}_N(\mu)\,\mathbf{u}_N(\mu) = \mathbf{f}_N(\mu) \quad \text{linear system in } N \times N$$

$\rightsquigarrow$ Decoder: $u_n(\mu) := D_N(\mathbf{u}_N(\mu)) := \mathbf{V}_N \mathbf{u}_N(\mu)$

Quasi-optimality property [under suitable hypothesis $\exists C_N > 0$]

$$\|u_h(\mu) - u_n(\mu)\| \leq C_N \inf_{\hat{u}_N \in V_N} \|u_h(\mu) - \hat{u}_N\|$$

Conclusion about the Reduced Basis Method

Successes

- Well suited for some classes of PDEs
- Theoretical guarantees!
- Matured technology to have in sight for a large category of problems!

"Reduced Basis Methods: Success, Limitations and Future Challenges", Ohlberger, Rave (2016)
"Reduced Basis Methods for Partial Differential Equations", Quarteroni, Manzoni, Negri (2015)

Some limitations

Difficulties when

- A large number of parameters $p \gg 1$, slow Kolmogorov widths decay of the inputs
- Some problems have a very slow Kolmogorov widths decay (even if p small)
  Famous case: Transport/convection dominated problems: $b \mapsto u(b)$ s.t. $\partial_t u + b \cdot \nabla u = f$

Going further ... need for non-linearity?

- Non-linear version of the Reduced Basis Method?
- Highly non linear models, Deep Learning?

3 - Our problem class of interest: parametric PDEs

3 Our problem class of interest: parametric PDEs

- Parametric PDEs settings
- The Reduced Basis Methods approach
- Nonlinear extension of the Reduced Basis Methods

Nonlinear Reduced Basis Method

The linear RBM model: looking for $u_N(\mu) \in V_N$

$$\mathbf{u}_h(\mu) \approx \mathbf{V}_N \mathbf{u}_N(\mu)$$

Nonlinear model

The general nonlinear RBM model : looking for $u_n(\mu) \in V_n$

$$\mathbf{u}_h(\mu) \approx \mathbf{V}_N \mathbf{f}(u_n(\mu))$$

Where $\mathbf{f} : \mathbb{R}^n \to R^N$ is a nonlinear map.

Nonlinear idea

We will consider the nonlinear model

$$\mathbf{u}_h(\mu) \approx \mathbf{V}^1_{n_1} \mathbf{u}_{n_1}(\mu) + \mathbf{V}^2_{n_2} \mathcal{N}\big(\mathbf{u}_{n_1}(\mu)\big), \ \ n_1 \ll N$$

Where $N = n_1 + n_2$ and $\mathbf{V}_N = \left[\mathbf{V}^1_{n_1} \middle| \mathbf{V}^2_{n_2}\right]$ and

$$\mathcal{N} : \mathbb{R}^{n_1} \to \mathbb{R}^{n_2}$$

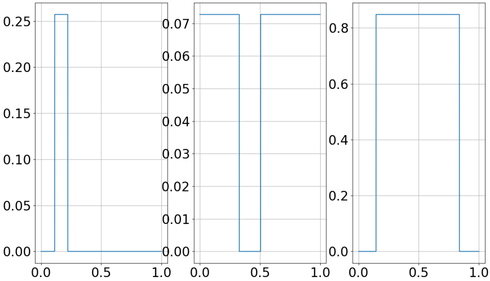
can be a “learnable” parametric nonlinear map.

The step functions case

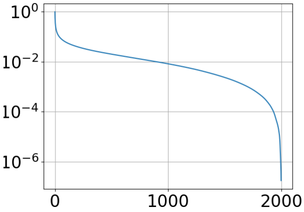
Nonlinear compressive reduced basis approximation for PDE's, Cohen, Farhat, Somacal, Maday, 2023

$$\mathscr{M} := \left\{ u(\mu) := b\mathbf{1}_{[a,a+\ell]} \pmod 1 : \mu = (a,b,\ell) \in [0,1]\times[0,1]\times[\ell_{min}, 1-\ell_{min}] \right\} \subset L^2(0,1)$$

- Not the solution manifold of an equation, but similar to transport of a step function
- Setting: probability distribution on $\mathscr{P}$: uniform



The step functions case



Key properties

- Low decay of Kolmogorov n-widths: $d_n(\mathscr{M}) \sim n^{-1/2}$: RBM will fail!
- The optimal SVD/POD modes are explicit: Fourier series

$$u(\mu) = \sum_{k\in\mathbb{N}} \alpha_k(\mu)\cos(2\pi kx) + \sum_{k\in\mathbb{N}^*} \beta_k(\mu)\sin(2\pi kx)$$

Explicitly $\alpha_0(\mu=(a,\ell,b)) = b\ell$, $\alpha_k(\mu) = \dots$, $\beta_k(\mu) = \dots$, with the interesting property that

$$\forall k \geq 2,\ \left(\alpha_k(\mu), \beta_k(\mu)\right) = \mathcal{F}\left(\alpha_0(\mu), \alpha_1(\mu), \beta_1(\mu)\right)$$

$\rightsquigarrow$ In the case of the step functions, $n_1 = 3$ while $N \sim 1/\varepsilon^2$

Manifold approximation

$\rightarrow$ Choices for $\mathbf{V}^1_{n_1}, \mathbf{V}^2_{n_2}, \mathcal{N}$?

Quadratic manifold approximation (*Operator inference with quadratic manifolds*, Geelen, Wright, Willcox, 2022)

$\mathbf{V}^1_{n_1}$ contains first n_1 left singular vectors of snapshot matrix S_{n_s}

$$\mathbf{u}_h(\mu) \approx \mathbf{V}^1_{n_1}\mathbf{u}_{n_1}(\mu) + \mathbf{V}^2_{n_2}\left[\mathbf{u}_{n_1}(\mu) \otimes \mathbf{u}_{n_1}(\mu)\right]$$

$$\mathbf{V}^2_{n_2} = \underset{\mathbf{V}^2_{n_2} \in \mathbb{R}^{N_h \times n_1(n_1+1)/2}}{\text{argmin}} \left(\sum_{j=1}^{n_s} \left\| \mathbf{u}_h(\mu^j) - \mathbf{V}^1_{n_1}\mathbf{u}_{n_1}(\mu^j) - \mathbf{V}^2_{n_2}\left[\mathbf{u}_{n_1}(\mu^j) \otimes \mathbf{u}_{n_1}(\mu^j)\right] \right\|_2^2 \right)$$

Neural networks approximation (*Neural networks PROM*, Barnett, Farhat, Maday, 2022)

$\mathbf{V}^1_{n_1}$ and $\mathbf{V}^2_{n_2}$ contains first $N = n_1 + n_2$ left singular vectors of snapshot matrix S_{n_s}

$$\mathbf{u}_h(\mu) \approx V^1_{n_1}\mathbf{u}_{n_1}(\mu) + \mathbf{V}^2_{n_2}\mathcal{N}_\theta(\mathbf{u}_{n_1}(\mu))$$

$\mathcal{N}_\theta : \mathbb{R}^{n_1} \rightarrow \mathbb{R}^{n_2}$ is a neural network.

$$\theta = \arg\min_{\theta \in \Theta} \sum_{j=1}^{n_s} \left\| \mathbf{u}_h(\mu^j) - \mathbf{V}^1_{n_1}\mathbf{u}_{n_1}(\mu^j) - \mathbf{V}^2_{n_2}\mathcal{N}_\theta(\mathbf{u}_{n_1}(\mu)) \right\|_2^2$$

Nonlinear projection problem

Difficulties & Questions

- In the case of a real PDE to solve, with the model

$$\mathbf{u}_h(\mu) \approx u_N(\mu) \in \mathcal{V}_n := \left\{ \mathbf{V}^1_{n_1}\mathbf{u} + \mathbf{V}^2_{n_2}\mathcal{N}(\mathbf{u}) \,:\, \mathbf{u} \in \mathbb{R}^{n_1} \right\}$$

  the Galerkin projection

$$\forall v_N \in \mathcal{V}_N, \quad a(u_N(\mu), v_N; \mu) = f(v_N; \mu)$$

  is a non linear problem!
  $\rightsquigarrow$ Gauss-Newton type algorithm, have to know to compute the gradient

- How to choose the splitting n_1 and n_2 ?

- What are the best choice of parametric model $\mathcal{N}$?

Ongoing work (with Anthony Nouy)

$\rightarrow$ Adaptive approach to determine n_1 with a search algorithm.
$\rightarrow$ Efficient choice for $\mathcal{N}$

Toy manifold

$$\mathcal{M} := \left\{ \left(\sin(t), \cos^2(t), \sin(3t)/4 \right), t \in [0, 2\pi] \right\}$$

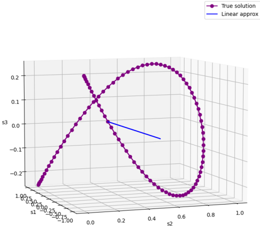


Figure: Linear approx. $n_1 = 1$

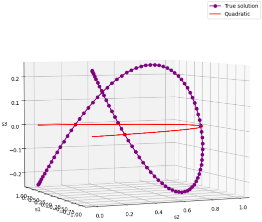


Figure: Quadratic approx. $n_1 = 1$

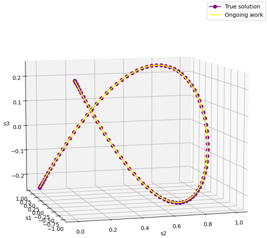


Figure: Proposed approx. $n_1 = 1$

4 - Physics Informed Networks

$$\text{formal PDE} \quad \begin{cases} \mathcal{P}(u) = f, & \Omega \quad \text{(Differential operator)} \\ \mathcal{B}(u) = g, & \partial\Omega \quad \text{(boundary condition)} \end{cases}$$

Natural and historical ideas

1. One has a parametric functions space $\mathscr{N} = \left\{u_\theta \,:\, \theta \in \Theta \subset \mathbb{R}^P\right\}$
2. One forms the PDE residual:
$$r(u) := \mathcal{P}(u) - f$$
3. One attempts to minimize the residue at some collocation points $(x_j) \in \Omega$ in the domain, and some at the boundary $(s_j) \in \partial\Omega$
$$\theta^\dagger := \arg\min_{\theta\in\Theta} \mathscr{L}(\theta) \quad \text{where } \mathscr{L}(\theta) := \mathscr{L}_r(\theta) + \mathscr{L}_b(\theta)$$
and were
$$\mathscr{L}_r(\theta) := \frac{1}{N_r}\sum_{j=1}^{N_r} |r(u_\theta)(x_j)|^2 \quad \mathscr{L}_b(\theta) := \frac{1}{N_b}\sum_{j=1}^{N_b} |\mathcal{B}u_\theta(s_j) - g(s_j)|^2$$
4. One expects that
$$u_{\theta^\dagger} \approx u$$

Questions about PINNs design 1/2

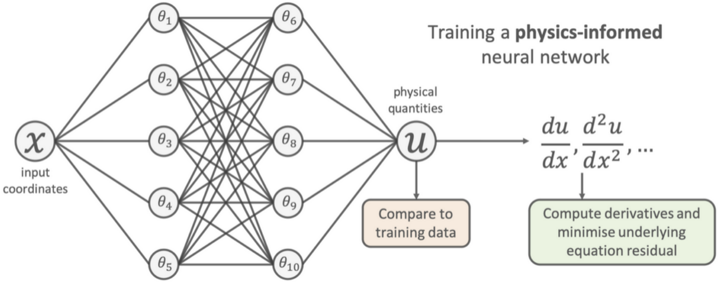


Figure: from Ben Mosseley webpage

A lot of design questions...

1. Expressivity of the class $\mathcal{N}$? Which Neural Network architecture?
2. What residue to consider? Collocation points, variational methods?
3. How to compute that residue for a $u_\theta \in \mathcal{N}$?
4. How solve the optimization problem? With theoretical guarantee ?

Questions about PINNs design 2/2

A lot of design questions...

1. Expressivity of the class $\mathcal{N}$? Which Neural Network architecture?
2. What residue to consider? Collocation points, variational methods?
3. How to compute that residue for a $u_\theta \in \mathcal{N}$?
4. How solve the optimization problem? With theoretical guarantee ?

Some possible answers

1. The most tested: Feed Forward Neural Network. But some tests with others.
   - *"Mathematical modeling on a Physics-Informed Radial Basis Function Network"*, Stenkin, Gorbachenko (2024)
   - *"An efficient neural network method with plane wave activation functions for solving Helmholtz equation"*, Cui, Wang, Xiang (2022)
2. Collocation points are the most spread. Alternative "variational PINNs" with variational approaches.
   - *"Variational Physics-Informed Neural Network For Solving Partial Differential Equations"*, Kharazmi, Zhang, Karniadakis (2019)
3. Thanks to the automatic differentiation algorithm!
4. No guarantee as far I know!

Three lessons learnt by practice

By practice and by

"An expert guide to training physics informed neural networks", Wang, Wang, Sankaran, Perdikaris (2023)

Three lessons learnt

1. Losses balanced: crucial for optimization
2. Statistical loss: random collocation point at each gradient step
3. "Curiculum learning": avoid getting stuck in very bad local minima

$1)$ Losses balanced

Problem

$\mathscr{L}_r$ and $\mathscr{L}_b$ can vary very differently during the optimization $\Rightarrow$ **one is not optimized**!

One introduces weights λ_r, λ_b:

$$\mathscr{L}(\theta) = \lambda_r \mathscr{L}_r(\theta) + \lambda_b \mathscr{L}_b(\theta) \tag{1}$$

Gradient descent: weights adaptation strategy

At step n

1. If $n \equiv 0 \ \ [n_0]$, update the weights

$$\widetilde{\lambda}_r := \frac{\|\nabla \mathscr{L}_r(\theta^n)\| + \|\nabla \mathscr{L}_b(\theta^n)\|}{\|\nabla \mathscr{L}_r(\theta^n)\|} \qquad \widetilde{\lambda}_b := \frac{\|\nabla \mathscr{L}_r(\theta^n)\| + \|\nabla \mathscr{L}_b(\theta^n)\|}{\|\nabla \mathscr{L}_r(\theta^n)\|}$$

   Moving average:

$$\lambda_r \longleftarrow \alpha \widetilde{\lambda}_r + (1-\alpha)\lambda_r \qquad \lambda_b \longleftarrow \alpha \widetilde{\lambda}_b + (1-\alpha)\lambda_b \tag{2}$$

2. Gradient descent step:

$$\theta_{n+1} = \theta_n - \eta \left[\lambda_r \nabla \mathscr{L}_r(\theta_n) + \lambda_b \nabla \mathscr{L}_b(\theta_n)\right]$$

2) Statistical loss

Problem

Some area of the domain are never explored! Consequence: overfitting!

Instead of

$$\mathscr{L}_r(\theta) := \frac{1}{N_r}\sum_{j=1}^{N_r} |r(u_\theta)(x_j)|^2, \qquad \mathscr{L}_b(\theta) := \frac{1}{N_b}\sum_{j=1}^{N_b} |\mathcal{B}u_\theta(s_j) - g(s_j)|^2$$

with fixed collocation points $(x_j) \in \Omega$, $(s_j) \in \partial\Omega$,

$$\mathbf{L}_r(\theta) := \mathbf{E}\Big|r(u_\theta)(X)\Big|^2 \quad \text{and} \quad \mathbf{L}_b(\theta) := \mathbf{E}\Big|\mathcal{B}u_\theta(S) - g(S)\Big|^2$$

where $X \sim \mathcal{U}(\Omega)$, $S \sim \mathcal{U}(\partial\Omega)$ are random variable
The actual losses are then random estimation of $\mathbf{L}_r(\theta)$ and $\mathbf{L}_b(\theta)$ at each gradient descent step, with (X_n) and (S_n) iid.

- Generate (X_n) iid $\mathcal{U}(\Omega)$, (S_n) iid $\mathcal{U}(\partial\Omega)$
- At the n-th gradient step, update of the looses:

$$\mathscr{L}_r^n(\theta) := \frac{1}{N_r}\sum_{j=1}^{N_r} |r(u_\theta)(X_n)|^2 \approx \mathbf{L}_r(\theta) \qquad \mathscr{L}_b^n(\theta) := \frac{1}{N_b}\sum_{j=1}^{N_b} |\mathcal{B}(u_\theta)(S_n)|^2 \approx \mathbf{L}_b(\theta)$$

3) "Curiculum learning"

Problem

Some PDE solutions are difficult to find for some values of the parameters whereas are simple for other values.
Example: Helmholtz at high/low frequency: $-\Delta u - k^2 u = 0$.

- moderate k: easy to learn
- large k: difficult to learn

Idea of curiculum learning

Principle: Learn a sequence of solutions corresponding to a sequence of parameter values: from simple to difficult!

Example: Helmholtz $\rightarrow$ progressively increase the frequency!

Interpretation: allow to avoid local very bad local minimum, by progressively leading the networks to a better local mimimum.

5 - Going further: Operator Learning - infinite dimensional parameter space

Operator learning settings

Target operator

$\mathcal{A}, \mathcal{U} \subset \mathscr{F}(\Omega, \mathbb{R})$ Banach / Hilbert of function spaces defined on $\Omega \subset \mathbb{R}^d$.

$$\mathcal{G}^\dagger : \mathcal{A} \to \mathcal{U}$$

Example

Solution $u \in \mathcal{U}$ of a PDE, with coefficients/source as $a \in \mathcal{A}$, e.g.

$$a \in L^\infty(\Omega) \mapsto u(a) \in H_0^1(\Omega) \quad \text{solution of} \quad \begin{cases} -\mathrm{div}(a \nabla u) + \sigma u = f, & \Omega, \\ u = 0, & \partial\Omega \end{cases}$$

Model: Family of parametrized operators, by $\theta \in \Theta$

$$\mathcal{G}_\theta : \mathcal{A} \to \mathcal{U}$$

Goal: Find an approximation of $\mathcal{G}^\dagger$ from our parameterized class of operator

$$\text{Find } \theta^\dagger \in \Theta \ / \ \mathcal{G}_{\theta^\dagger} \approx \mathcal{G}^\dagger$$

Operator learning: the supervised learning settings

$\rightarrow$ Given an probability measure on $\mathcal{A}$ of the data $a \sim \mu$.

Error to control

$$\|\mathcal{G}^\dagger - \mathcal{G}_\theta\|^2_{L^2_\mu(\mathcal{A},\mathcal{U})} := \int_{\mathcal{A}} \|\mathcal{G}^\dagger(a) - \mathcal{G}_\theta(a)\|^2 d\mu(a)$$

Data

Sample of "examples" $a^j \sim \mu$ together with their associated solution $u^j := \mathcal{G}^\dagger(a_j)$, for $1 \leq j \leq N$

Empirical error

Approximation of the target (and inaccessible) error

$$\|\mathcal{G}^\dagger - \mathcal{G}_\theta\|_{L^2_\mu(\mathcal{A})} \approx \frac{1}{N}\sum_{j=1}^{N} \|u^j - \mathcal{G}_\theta(a_j)\|^2_{\mathcal{U}}$$

Learning problem

From the data $\{a^j, u^j\}_{j=1}^N$, solve the optimization problem

$$\theta^\dagger = \arg\min_{\theta\in\Theta} \frac{1}{N}\sum_{j=1}^{N} \|u^j - \mathcal{G}_\theta(a^j)\|^2_{\mathcal{U}}$$

The need for discretization

Remark: the previous learning problem is still abstract! The functions $a^j \in \mathcal{A}$ and $u^j \in \mathcal{U}$ are not discretized! Same problem for $\mathcal{G}_\theta : \mathcal{A} \to \mathcal{U}$

Input / output discretization

- Point discretization: we can assume that we know the input functions $a \in \mathcal{A}$ discrete point set $D = \{x_1, \ldots, x_{d_\mathcal{A}}\} \subset \Omega$
- Mesh discretization
- Spectral discretization

$$\begin{array}{ccc} \mathcal{A} & \xrightarrow{\mathcal{G}^\dagger} & \mathcal{U} \\ \downarrow \Psi_h & & \downarrow \Phi_h \\ \mathcal{A}_h & \xrightarrow{\mathcal{G}_h^\dagger} & \mathcal{U}_h \end{array} \quad \text{and} \quad \begin{array}{ccc} \mathcal{A} & \xrightarrow{\mathcal{G}_\theta} & \mathcal{U} \\ \downarrow \Psi_h & & \downarrow \Phi_h \\ \mathcal{A}_h & \xrightarrow{\mathcal{G}_{h,\theta}} & \mathcal{U}_h \end{array}$$

First ideas of learning operators

Wish features list

- **The model is a function:** $\mathcal{G}_\theta$ can be evaluated at any point of the domain
- **The data are functions:** the model can be trained on any discretization of its input or output training set functions (mesh invariance)
- **"Convergence":** the model converges toward a continuum operator as the discretization is refined
- **"No aliasing":** treatment by the model of various discretization of a function produced the same coherent output function
  - *"Representation Equivalent Neural Operators: a Framework for Alias-free Operator Learning"*, Bartolucci, Bézenac, Raonic, Molinaro, Mishra, Alaifari (2023)
- **"Equivarience":** the operator model $\mathcal{G}_\theta$ as the same equivariance as $\mathcal{G}^\dagger$, by construction, for any $\theta \in \Theta$.
  Equivariance: $\mathcal{G}^\dagger(ga) = \rho(g)\mathcal{G}^\dagger(a)$, $G \times \mathcal{A} \to \mathcal{A}$ a group action, $G' \times \mathcal{U} \to \mathcal{U}$ another group action, and $\rho : G \to G'$ group morphism.
  - *"Equivarient Graph Neural Operator for Modeling 3D Dynamics"*, Xu, Han, Lou, Kossaifi, Ramanathan, Azzizadenesheli, Leskovec, Ermon, Anandkumar (2024)

PCA-Net

“First” natural idea

- Represent on a basis the input functions and the output functions + truncate
- Learn the projected operator!

“Operator learning with PCA-Net: upper and lower complexity bounds”, Lanthaler (2023)

$$\begin{tikzcd} \mathcal{A} \arrow[rr, "\mathcal{G}^{\dagger}"] \arrow[d, "\Psi_n"] & & \mathcal{U} \\ \mathcal{A}_n \cong \mathbb{R}^n \arrow[rr, "\mathcal{G}_{\theta}^{\hat{n},m}"] & & \mathbb{R}^m \cong \mathcal{U}_m \arrow[u, "\Phi_m"'] \end{tikzcd}$$

and $G_{\theta}^{n,m} : \mathbb{R}^n \longrightarrow \mathbb{R}^m$ Deep Forward Neural Network $\theta \in \Theta$.

- PCA for input (encoding): $\mathcal{A}_n := \mathrm{span}\{\psi_1, \dots, \psi_n\}$ (orthonormal family)

$$\Psi_n(a) := \big(\langle a, \psi_j \rangle\big)_{1 \le j \le n}$$

- PCA for ouput (decoding): $\mathcal{U}_m := \mathrm{span}\{\phi_1, \dots, \phi_m\}$ orthonormal family

$$\Phi_n\big((\mathbf{u}_j)_{1 \le j \le m}\big) := \sum_{j=1}^{n} \mathbf{u}_{1 \le j \le m}\, \phi_j$$

DeepONet: Learn also the bases!

"Deeponet: Learning nonlinear operators for identifying differential equations based on the universal approximation theorem of operators", Lu, Jin, Karniadakis (2019)

$$\forall a \in \mathcal{A}, \quad \mathcal{G}_\theta(a) : x \in \Omega \longmapsto \sum_{k=1}^{K} \alpha_k(\mathcal{S}[a];\theta_a)\,\psi_k(x;\theta_b)$$

where

- $\mathcal{S}[a] : a \in \mathcal{A} \mapsto \mathbb{R}^{N_s}$: sampling operator

$$\mathcal{S} : a \in \mathcal{A} \longmapsto \mathcal{S}[a] := \left[a(x_1), \ldots, a(x_{N_s})\right] \in \mathbb{R}^{N_s}$$

- **Non-linear coefficients:** $\alpha_k : \mathbb{R}^{N_s} \times \Theta_\alpha \to \mathbb{R}$: is a Feed Forward Neural Networks (parameters set Θ_α)
- **Learnt "basis":** $\psi_k : \Omega \times \Theta_\psi \to \mathbb{R}$: is a Feed Forward Neural Network (parameter set Θ_ψ)
- Total parameters: $\theta = (\theta_\alpha, \theta_\psi) \in \Theta_\alpha \times \Theta_\psi$

A universal theorem is available.

Going further: Neural operators

Idea

As in Feed Forward Neural Network, but with functions
$\rightsquigarrow$ Composition of linear operators (with kernel) and non-linearity!

$$\mathcal{G}_\theta := \Pi \circ \sigma(\mathcal{W}_T + \mathcal{K}_T) \circ \ldots \circ \sigma(\mathcal{W}_1 + \mathcal{K}_1) \circ \mathcal{L}$$

- $\mathcal{L}, \Pi$ (pointwise operators) $L : \mathbb{R}^{d_{in}} \to \mathbb{R}^{d_L}, \ \ \Pi : \mathbb{R}^{d_L} \to \mathbb{R}^{d_{out}}$,
- $\mathcal{W}_t$ are pointwise linear operators parameterized as matrices $W_t \in \mathbb{R}^{d_L \times d_L}$
- $\mathcal{K}_t$ are integral operators,

$$(\mathcal{K}_t v)(x) = \int_\Omega \kappa_\theta^t\big(x, y, a(x), a(y)\big)\, v(y)\, dy,$$

  $\kappa_\theta^t\big(x, y, a(x), a(y)\big) \in \mathbb{R}^{d_L \times d_L}$ a function parametrized by a Neural Network,
- σ nonlinear activation function.

Learning operators with graphs (Graph Kernel network)

Problem

$\rightarrow$ How to **discretize integral** operators at **reasonable cost**?

Some approaches

- Graph Kernel Neural Networks
- Multipole-Graph Kernel Neural Network
- Fourier Neural Networks

Graph Kernel Neural Networks

Construct graph on spatial domain Ω

- Node of the graph: x_i
- Neighbors of x_i: $N(x_i)$

The "integral" operator at point x_i is approached by a sum over the neighbors of x_i:

$$(\mathcal{K}_t v)(x_i) = \text{``} \int_{B(x,r)} \kappa_\theta^t(e(x_i,y))v(y)dy \text{''} \approx \frac{1}{|N(x_i)|} \sum_{x_j \in N(x_i)} \kappa_\theta^t(e(x_i,x_j))v(x_j)$$

- edge features $e(x,y) = (x,y,a(x),a(y))$
- κ_θ^t is parametrized by a neural network

Fourier neural operator

Consider integral operator

$$(\mathcal{K}v)(x) = \int_\Omega \kappa_{\theta'}(x,y,\mu)v(y)dy$$

$\rightarrow$ Assume kernel is of form $\kappa(x,y,\mu) = \kappa(x-y)$

$$(\mathcal{K}v)(x) = \int_\Omega \kappa(x-y)v(y)dy = \mathcal{F}^{-1}\big(\underbrace{\mathcal{F}(\kappa)}_{F(k)\cdot\mathcal{F}}(v)(k)\big)(x)$$

In practice

- Use Fast Fourier Transform for $\mathcal{F}$,
- Truncate Fourier series with maximum number of modes k_{max},

$$F_{\theta'} \in \mathbb{C}^{k_{max}\times d_L \times d_L}, \;\; \mathcal{F}(v) \in \mathbb{C}^{k_{max}\times d_L},$$

- Parameterization of $\kappa = \kappa_\theta$: in Fourier space for $1 \le k \le k_{max}$, $F_\theta(k) := \theta_k$ with $\theta_k \in \mathbb{C}^{d_L\times d_L}$ trainable parameters

We consider different databases with growing complexity of the solution. $(k = 10, k = 40, k = 90)$

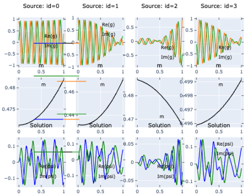


Figure: **DATABASE 1**

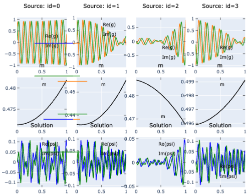


Figure: **DATABASE 2**

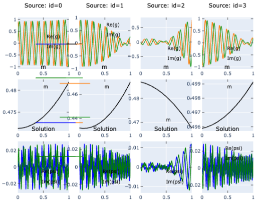


Figure: **DATABASE 3**

→ GNO was trained on coarse grid $(s = 64)$
→ FNO trained on $s = 512$
→ Increase depth in FNO is crucial for *difficult* cases

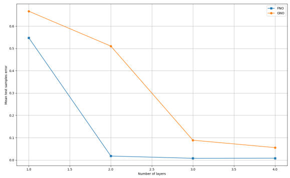


Figure: DB1

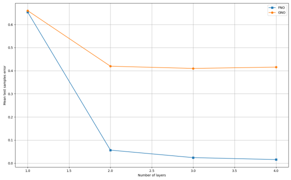


Figure: DB2

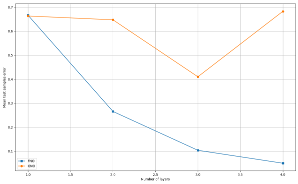
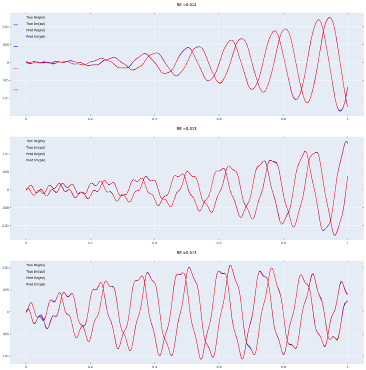


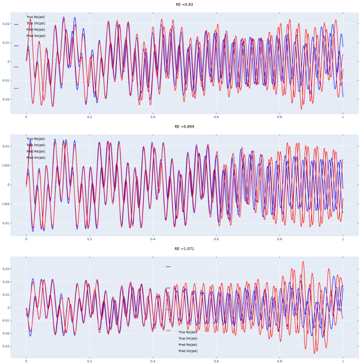
Figure: DB3

	$T=1$	$T=2$	$T=3$	$T=4$		$T=1$	$T=2$	$T=3$	$T=4$
DB1(FNO)	0.547	0.01	0.007	0.007	**DB1(GNO)**	0.666	0.510	0.088	0.055
DB2(FNO)	0.655	0.056	0.024	0.015	**DB2(GNO)**	0.660	0.419	0.410	0.415
DB3(FNO)	0.666	0.265	0.103	0.049	**DB3(GNO)**	0.663	0.647	0.650	0.682

Numerical results

→ Best and worst predictions on **DATABASE 3** with FNO

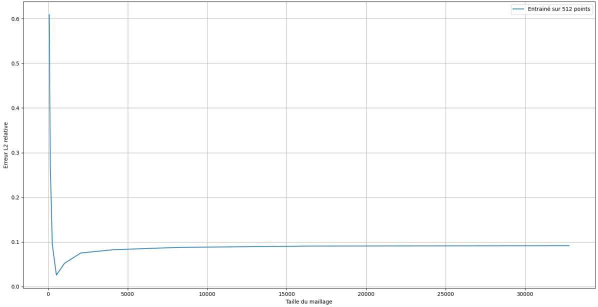




→ Same observations on other databases.

Numerical results

$\rightarrow$ Generalization of FNO (DB3) on finer grids



Conclusion and openings

A theoretical framework of approximation / learning

- Fast growing field of neural network approximation capability
- What are the low hidden dimensions which can be explored by neural network and other non-linear tools (tensors, ...) ?

Do not forget the “old” AI!

- The Reduced Basis Method works for a broad class of problems
- Non linear Model Order Reduction (MOR) are still in active developpment

A lot to do with new AI

- Still a lot to explore in Deep Learning application to scientific computing
- Need for researchers from numerical analysis to hybridate!

thanks!

Thank you for you attention!!

Any questions?